PROJECT REPORT

Lightweight Retrieval-Time Detection of Indirect Prompt Injection in Local RAG Systems

A partially fine-tuned DistilBERT detector that gates clause-level sanitization: complete project record and reference material for the conference paper

Prepared for	[Guide name], Project Guide
Prepared by	[Student name(s)]
Institution	[Department], [Institution]
Date	29 October 2026
Target venue	COMSNETS 2027, Bengaluru, 5–9 January 2027
Base paper	J. Yi *et al.*, "Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models" (BIPIA), ACM KDD 2025, pp. 1809–1820, doi: 10.1145/3690624.3709179
Status	Layer 1 detector trained and evaluated on 20,000 held-out chunks; end-to-end RAG benchmark not yet complete

Contents

Executive Summary **3**

1 Introduction **4**
1.1 Context 4
1.2 Problem Statement 4
1.3 Base Paper: BIPIA (KDD 2025) 4
1.4 Objectives 5
1.5 Contributions 5
1.6 Scope and Organisation of this Report 5

2 Background and Related Work **6**
2.1 Indirect Prompt Injection in RAG 6
2.2 Detection-Based Defenses 6
2.3 Prevention, Sanitization and Layered Defenses 6
2.4 Benchmarks for RAG and Agent Security 6
2.5 Positioning 6
2.6 What the Project Can and Cannot Claim 7

3 Threat Model and System Architecture **7**
3.1 Threat Model 7
3.2 Architecture 8
3.3 Design Goals 9
3.4 Components and Training Status 9

4 Benchmark Dataset **10**
4.1 Design Principles 10
4.2 Corpus Composition 10
4.3 Attack Families 10
4.4 Hard Benign Negatives 11
4.5 Partitions and Isolation 11
4.6 Reproducibility and Automated Checks 12
4.7 External Reference Data 12
4.8 From the Legacy Corpus to the 100k Benchmark 12

5 Layer 1: Detector Design and Training **12**
5.1 Why a Compact Encoder 12
5.2 Windowed Scoring 12
5.3 Partial Fine-Tuning and Parameter Budget 13
5.4 Training Data 13
5.5 Training Configuration 14
5.6 Checkpoint and Threshold Selection 14
5.7 Compute Environment 14

6 Evaluation Methodology **14**
6.1 Protocol 14
6.2 Metrics 15
6.3 Confidence Intervals 15
6.4 Caveat on Independence 15

7 Detection Results **15**
7.1 Regular Test: Seen Families 15
7.2 Challenge Set: Withheld Families 16
7.3 Per-Family Analysis 17

7.4 Why the Three Families Are Missed ... 18
7.5 Operating Point and the Cost of Errors ... 18
7.6 Summary of Findings ... 18
8 Defense Layer (Layer 2) and End-to-End Harness ... 19
8.1 Detector-Gated Clause Sanitization ... 19
8.2 Preliminary Verification ... 19
8.3 End-to-End Evaluation Harness ... 20
8.4 Status ... 20
9 Implementation, Tools and Reproducibility ... 21
9.1 Software Stack ... 21
9.2 Code Organisation ... 21
9.3 Inspection Workbench ... 21
9.4 Engineering Safeguards and Tests ... 22
9.5 Reproducing the Results ... 22
9.6 Output Files ... 22
10 Limitations ... 22
11 Recommended Next Experiments ... 23
12 Guidance for Writing the Conference Paper ... 24
12.1 Venue Facts: COMSNETS 2027 ... 24
12.2 Recommended Framing ... 24
12.3 Suggested Structure and Page Budget ... 25
12.4 Figures and Tables for the Paper ... 26
12.5 Claims and Wording ... 26
12.6 Items to Verify Before Submission ... 27
12.7 Anonymisation and Artifact Availability ... 27
13 Conclusion ... 27
References ... 27
Appendix A: Reproducing the Figures ... 30
A.1 Figures in this Report (Python) ... 30
A.2 MATLAB: Step by Step ... 30
A.3 Listing: make_report_figures.m ... 31
A.4 Listing: wilson_ci.m ... 33
Appendix B: Equations in LaTeX ... 33
Appendix C: Recorded Numbers and Derivations ... 34
C.1 Confusion Matrices (Rows: Actual; Columns: Predicted) ... 34
C.2 Derived Quantities ... 34

Executive Summary

This report is reference material for the conference paper on the project *A Lightweight Hybrid Defense Framework Against Indirect Prompt Injection in RAG Systems*. It concentrates on **detection (Layer 1)**, the trained and fully evaluated part, and covers the **defense layer (Layer 2)** briefly.

Problem and approach. In indirect prompt injection, an attacker hides instructions in documents that a retrieval-augmented generation (RAG) system later passes to the language model. The base paper, BIPIA (ACM KDD 2025) [1], found all 25 evaluated LLMs vulnerable, and its strongest defense fine-tunes the generator, which a small CPU-only local deployment cannot afford. This project screens every retrieved chunk *before* generation with a small DistilBERT classifier (Layer 1) and sends only flagged chunks to a rule-based clause sanitizer (Layer 2), so the defense never calls a generative model.

What has been done.

- **A complete local RAG pipeline:** MiniLM retrieval with Chroma, the two defense layers and a Llama 3.2 3B generator served by Ollama, plus a four-mode evaluation harness, a web workbench and unit tests.
- **A 100,000-chunk synthetic benchmark:** 50,000 policies, each clean and injected; 10 topics; 5,000 divisions; 32 attack families, 8 of them withheld for a challenge set.
- **One trained module,** the Layer 1 DistilBERT detector. Its last block and head were fine-tuned (7,680,002 of 66,955,010 parameters, 11.47%) on a stratified 6,000-chunk sample for 3 epochs on a laptop CPU. Epoch 1 and $\tau = 0.175$ were selected on validation data.
- **Evaluation and figures:** the detector was evaluated on 20,000 held-out chunks and the sanitizer checked on the earlier 200-chunk corpus. Print-ready figures come with MATLAB and Python scripts (Appendix A).

Table 1: Key results at a glance (95% Wilson intervals in brackets)

Quantity	Regular test (10,000 chunks, 24 seen families)	Challenge (10,000 chunks, 8 withheld families)
Accuracy	100.00% [99.96, 100.00]	98.09% [97.80, 98.34]
Recall (detection rate)	100.00% [99.92, 100.00]	96.18% [95.61, 96.68]
False-positive rate	0.00% (upper bound 0.077%)	0.00% (upper bound 0.077%)
Errors	none	191 missed injections, 0 false alarms
Weakest attack family	—	multilingual: 76.36% recall (148 of 626 missed)

Notes for the guide

- **Venue timing.** The COMSNETS 2027 main-track deadline (31 August 2026) has passed; reported open tracks are the Graduate Forum (about 8–9 November; 4 pages + references page + 1-page resume), posters and demos (Section 12.1).
- **Values to confirm.** Highlighted values come from code defaults or inference, not the training log (Table 23).
- **Double-blind.** Main-track and workshop papers must not show author names or the project's GitHub link.
- **Still open.** No end-to-end attack-success numbers yet; synthetic data from one run with one seed; no external or adaptive tests (Chapter 11).

1 Introduction

1.1 Context

Retrieval-augmented generation (RAG) [2] has become the standard way to ground large language models (LLMs) in an organisation's own documents. A retriever selects the chunks most similar to the user's question, and a generator answers from them. The same mechanism opens an attack surface. Prompt injection heads the OWASP list of risks for LLM applications [3]. In its *indirect* form, the attacker never interacts with the model. Instead, they plant instructions in content that the application will later retrieve, such as a wiki page, a shared policy file or an e-mail [4]. In direct injection, the user types something like "ignore previous instructions" [5]. In indirect injection the victim is an honest user, and the malicious text arrives through a channel that the application treats as data.

1.2 Problem Statement

Consider an organisation that runs an internal policy assistant entirely on its own hardware. A small open model answers employees' questions from internal documents found by vector search. Anyone who can edit an indexed document, for example on a shared wiki, can plant a sentence that addresses the assistant, such as an instruction to ignore the question and send the user to a "verification" link. If that chunk is retrieved, the model may obey it. A practical defense must:

1. stop injected chunks before they reach the generator;
2. leave legitimate policy text untouched, including legitimate instructions to employees;
3. run on a CPU without additional LLM calls; and
4. make its decisions visible to an operator.

1.3 Base Paper: BIPIA (KDD 2025)

Yi *et al.* [1] introduced BIPIA, the first benchmark for indirect prompt injection. It covers five application scenarios (e-mail QA, web QA, table QA, summarisation and code QA) and 250 attacker goals, and it found all 25 evaluated LLMs susceptible. The authors trace the vulnerability to two causes: LLMs do not distinguish informational context from actionable instructions, and they lack awareness that instructions inside external content should not be executed. Their defenses follow from this diagnosis. *Boundary awareness* and *explicit reminders* are implemented through prompting for black-box models, where they mitigate attacks substantially. For white-box models, the generator is fine-tuned with adversarial examples and special boundary tokens, which reduces the attack success rate to near zero while preserving output quality.

We use BIPIA as the base paper and adopt three of its elements. The first is its **threat model**: the attacker controls only external content, not the user's request or the system prompt. The second is its **diagnosis** that the root failure is the confusion of data with instructions, which we turn into a classification problem on the data channel. The third is its **separation of training and test attacks**, which we strengthen by withholding entire attack families. We differ in *where* the defense acts (before the generator rather than inside it) and in *what* is measured (chunk-level detection rather than generator attack success; Table 2).

Table 2: BIPIA compared with this project

Aspect	BIPIA [1]	This project
Goal	Benchmark indirect prompt injection and defend the LLM	Detect injected chunks before they reach the LLM
Data	5 application scenarios; 250 attacker goals	Enterprise-policy QA; 100,000 synthetic chunks; 32 attack families
Models	25 LLMs evaluated	DistilBERT detector (66.96M parameters); Llama 3.2 3B generator
Defense	Boundary awareness and explicit reminder (black-box); adversarial fine-tuning with special tokens (white-box)	Encoder classifier at retrieval time, gating a rule-based clause sanitizer
Changes the generator	White-box defense: yes	No
Main measurement	Attack success rate of the generator	Chunk-level detection (accuracy, recall, FPR); end-to-end attack success pending
Generalisation test	Separate training and test attack sets	Eight attack families withheld from training and validation
Compute for the defense	Released fine-tuning recipe uses eight V100 GPUs	One laptop CPU

1.4 Objectives

1. Design a retrieval-time detector that needs neither a GPU nor a generative call.
2. Build a benchmark that tests generalisation to attack families never seen in training.
3. Measure detection accuracy, false-positive rate and per-family behaviour with uncertainty estimates.
4. Integrate the detector with a sanitizer that preserves legitimate content, inside an inspectable pipeline.

1.5 Contributions

- **Benchmark.** 100,000 synthetic enterprise-policy chunks (10 topics, 5,000 fictional divisions, 50,000 policies). Each policy appears as a clean and an injected chunk, across 32 attack families (96 templates) with hard benign negatives. Eight families and 1,000 divisions are held out for evaluation (Chapter 4).
- **Detector.** A windowed DistilBERT classifier [6] with max-over-windows scoring. Only the last block and the head (7.68M parameters) are fine-tuned, on 6,000 chunks on a CPU, and the threshold is set on validation data under a 2% false-positive constraint (Chapter 5).
- **Held-out evaluation.** On 20,000 chunks: no errors on known families, 96.18% recall with zero false positives on withheld families, and a per-family analysis that isolates multilingual and dependency-trap injections as the main blind spots (Chapter 7).
- **Inspectable pipeline.** The detector gates a clause sanitizer, with a four-mode evaluation harness and a preliminary verification of the sanitizer (Chapters 8–9).

1.6 Scope and Organisation of this Report

Chapter 2 reviews related work and positions the project; Chapter 3 defines the threat model and architecture; Chapter 4 describes the benchmark; Chapter 5 presents the detector and its training; Chapters 6 and 7 give the evaluation method and results. Chapter 8 covers the defense layer briefly, and Chapter 9 the implementation. Chapters 10 and 11 list limitations and next experiments, and Chapter 12

collects guidance for writing the paper. The appendices explain how to regenerate every figure in MATLAB or Python and list the raw numbers.

2 Background and Related Work

2.1 Indirect Prompt Injection in RAG

Greshake *et al.* [4] showed that attacker-controlled web pages, e-mails and documents can hijack LLM-integrated applications. Liu *et al.* [7] formalised prompt injection and benchmarked attacks and defenses across tasks and models. BIPIA [1] then provided a dedicated benchmark for the indirect setting (Section 1.3). Injection should be distinguished from *knowledge corruption*: PoisonedRAG [8] inserts crafted false passages that steer the answer without any instruction. That threat is out of scope here, because a detector of instructions cannot recognise a well-formed false statement.

2.2 Detection-Based Defenses

Detectors classify inputs as injected or benign. PIGuard [9] trains a guardrail classifier and shows that a bias toward trigger words causes over-defense on benign inputs, which it measures with its NotInject set. PromptShield [10] argues that detectors must be judged at deployable, very low false-positive rates. DataSentinel [11] fine-tunes an LLM-based detector through a minimax game against adaptive attacks. Akinrele and Gowda [12] show that detector performance depends strongly on the deployment regime. Human-written attack corpora such as Tensor Trust [13] supply diverse injections. Our detector follows the supervised-classifier line. It takes up PIGuard's concern with over-defense through hard benign negatives and PromptShield's emphasis on low false-positive rates through a validation constraint. It adds a family hold-out protocol in a RAG setting.

2.3 Prevention, Sanitization and Layered Defenses

Prompt-level defenses such as Spotlighting [14] mark the provenance of external text by delimiting, datamarking or encoding it. *Training-level* defenses, namely StruQ [15], SecAlign [16] and the instruction hierarchy [17], teach the model to prioritise trusted instructions. *System-level* designs such as Task Shield [18] and CaMeL [19] check task alignment or separate control flow from data flow. PromptArmor [20] prompts an off-the-shelf LLM to detect and remove injected text, which is the objective closest to our sanitizer, at the cost of an LLM call. Layered designs also exist. Spotlight-Guard [21] combines spotlighting, LLM-based detection and integrity checks for agents, and Raghuraj *et al.* [22] benchmark hybrid RAG defenses across models.

2.4 Benchmarks for RAG and Agent Security

InjecAgent [23] and AgentDojo [24] evaluate injections against tool-using agents, and AgentDyn [25] adds dynamic tasks in which third-party instructions can be legitimate. SafeRAG [26] covers several RAG-specific threats, and Hidden-in-Plain-Text [27] targets social-web content in RAG. A recent survey [28] organises the field. Our benchmark is narrower in task (enterprise-policy QA) but is designed specifically to test whether a detector generalises to attack families withheld from training.

2.5 Positioning

Table 3 compares where representative defenses act and what they require. The design point of this project is a defense that leaves the generator unchanged and makes no generative call, which suits CPU-only local deployment.

Table 3: Where representative defenses act and what they require

Defense	Acts on	Changes generator	LLM call in defense
BIPIA black-box [1]	Prompt	No	No
BIPIA white-box [1]	Generator weights	Yes	No
Spotlighting [14]	Prompt	No	No
StruQ, SecAlign [15], [16]	Generator weights	Yes	No
PIGuard [9]	Input text (classifier)	No	No
DataSentinel [11]	Input text (LLM detector)	No	Yes
PromptArmor [20]	Input text (LLM removal)	No	Yes
This project	**Retrieved chunks (encoder + rules)**	**No**	**No**

2.6 What the Project Can and Cannot Claim

Because several elements of the design already have prior art, the paper's claims must be precise. Table 4 summarises which statements the evidence supports.

Table 4: Claims supported and not supported by the current evidence

Statement	Assessment
"We discovered indirect prompt injection in RAG."	Not supported: established by [4] and [1].
"First hybrid or dual-layer defense."	Not supported: layered defenses exist [21], [22].
"First to remove injections while keeping useful content."	Not supported: PromptArmor removes injected text [20].
"A CPU-trained detector reaches 96.18% recall with zero false positives on eight withheld families of this benchmark."	Supported, for this synthetic benchmark.
"The defense stage makes no generative-model call."	Supported by the design.
"Clause sanitization preserves legitimate imperative policy text."	Partly supported (legacy checks only); needs independent annotation.
"The system reduces end-to-end attack success."	Not yet measured.
"Faster or more accurate than the state of the art."	Not established: no matched comparison yet.
"The defense guarantees protection."	Not supported: misses and adaptive attacks remain possible.

3 Threat Model and System Architecture

3.1 Threat Model

- **Setting.** An internal assistant answers employees' policy questions from an indexed document collection.
- **Trust boundary.** The application code, the system prompt and the user's question are trusted. Every retrieved chunk is untrusted, because an attacker may author or edit documents that are later indexed.
- **Attacker goal.** Make the generator deviate from the user's task, for example by:
  - emitting an attacker-chosen string;

  - redirecting the user to an external link;
  - dictating the answer format;
  - impersonating a system or developer message;
  - leaking data through markup.
- **Attacker capability.** The attacker can place instructions anywhere in a chunk and can use formatting tricks (XML, JSON, YAML, comments), social engineering (urgency, authority, politeness) and obfuscation (other languages, Unicode look-alikes, encodings).
- **Assumptions and exclusions.** The attacker is non-adaptive: they know neither the detector's weights nor its threshold. Adaptive attacks, false declarative statements [8] and tool-using agents are out of scope.

3.2 Architecture

Figure 1 shows the pipeline. Every retrieved chunk passes through the detector before it can reach the prompt:

1. The question is embedded with all-MiniLM-L6-v2 [29], [30] (384 dimensions, 22.7M parameters, frozen).
2. A Chroma vector store returns the top-k chunks.
3. Layer 1 (Chapter 5) assigns each chunk an injection score; chunks scoring below $\tau = 0.175$ pass unchanged.
4. Flagged chunks are sanitized clause by clause by Layer 2 (Chapter 8); they are edited, not discarded.
5. The context is assembled with an explicit reminder that retrieved text is untrusted reference material, and a local Llama 3.2 3B model [31] served by Ollama generates the answer.
6. Every run records retrieval scores, detector scores, removed and retained clauses, the processed context, the answer and the latency.

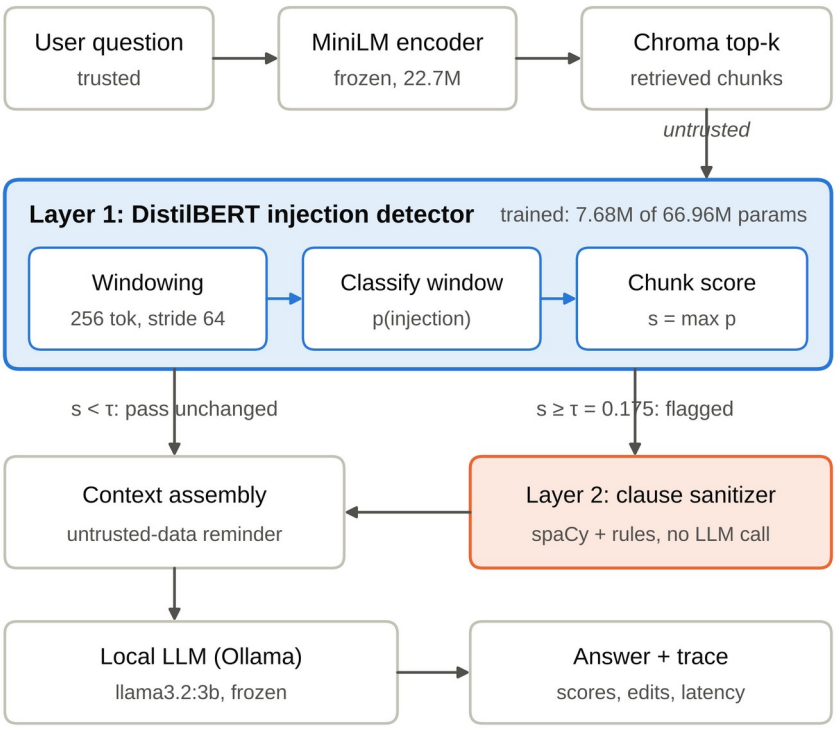


Figure 1: Detection-centred pipeline. Layer 1, the only trained component, screens every retrieved chunk before generation. Only flagged chunks reach the rule-based Layer 2 sanitizer, so the defense makes no LLM call.

3.3 Design Goals

- **G1, no generative call inside the defense.** The cost of the defense must not scale with LLM inference.
- **G2, CPU-only.** Training and inference must run on a laptop CPU.
- **G3, preserve benign evidence.** Very few false alarms, and no loss of policy facts when a chunk is sanitized.
- **G4, inspectability.** Every decision (score, threshold, removed clause and its reason) is visible to an operator.

3.4 Components and Training Status

Table 5 lists every component of the pipeline. **Exactly one module is trained in this project: the Layer 1 detector.** This report presents its single recorded checkpoint (29 October 2026). The retriever and the generator are used as released, the spaCy parser is pretrained, and the sanitizer rules and the baselines involve no learning.

Table 5: Pipeline components, parameter counts and training status

Component	Model / implementation	Parameters	Status
Retriever encoder	sentence-transformers all-MiniLM-L6-v2	22.7M	pretrained, frozen
Vector index	Chroma (persistent)	—	built, not trained
Layer 1 detector	**DistilBERT-base-uncased + classification head**	**66.96M (7.68M fine-tuned)**	**fine-tuned (this project)**
Layer 2 parser	spaCy en_core_web_sm	—	pretrained, used as-is
Layer 2 rules	hand-written contextual patterns	—	no learning
Generator	Llama 3.2 3B via Ollama	3.21B	pretrained, frozen
Baselines	delimiter prompt; LLM-judge prompt	—	no learning

4 Benchmark Dataset

4.1 Design Principles

Public prompt-injection datasets rarely provide an enterprise document, its clean counterpart, the question it answers and the fact that must survive sanitization. We therefore generated a synthetic benchmark under five principles:

1. **Paired chunks.** Every policy appears as a clean and an injected chunk, so the label depends only on the injected content.
2. **Answerable facts.** Each policy carries specific numeric facts and a benchmark question with its expected answer (evaluation-only metadata).
3. **Hard negatives.** Benign chunks contain legitimate instructions, quoted attack phrases, markup and inert target strings.
4. **Held-out families.** Eight attack families never appear in training or validation.
5. **Isolation.** Evaluation partitions use policies and divisions never seen in training.

4.2 Corpus Composition

The corpus simulates the policy library of a large organisation. It covers ten topics: refunds, IT security, HR leave, financial reports, server configurations, travel expenses, data retention, vendor procurement, customer support and business continuity. There are 5,000 fictional divisions and 50,000 policies. Rendering every policy twice gives 100,000 chunks. The generator is seeded, and its size scales in multiples of 200 rows to keep topics balanced and the 70/10/10/10 split exact.

4.3 Attack Families

Injections are instantiated from 96 templates grouped into 32 families (Table 6). They range from overt commands and role-play, through structural spoofing in XML, JSON, YAML, Markdown and code comments, to social-engineering lures and manipulation of the answer's prefix, format or routing. The eight withheld families test styles that differ from training. Some are subtle (indirect completion, reasoning lures, polite requests, few-shot imitation, dependency traps) and some are obfuscated (multilingual instructions, Unicode spoofing, encoded instructions).

Table 6: Attack families in the benchmark

Split	Families
Seen (24): training, validation, regular test	direct imperatives, role-play, XML context breakouts, hidden comments, evaluation lures, tool spoofing, conversation injection, footer injection, developer spoofing, Markdown instructions, JSON spoofing, YAML spoofing, urgency, reward lures, redirection, answer-prefix hijacking, compliance spoofing, translation instructions, routing hijacks, format hijacks, nested XML, fake memory, code comments, authority claims
Withheld (8): challenge only	indirect completion, dependency traps, reasoning lures, polite requests, few-shot imitation, multilingual instructions, Unicode spoofing, encoded instructions

The dataset notes name the eighth withheld family "encoded instructions" in one place and "output-protocol lures" in another; confirm against the generator's family list.

4.4 Hard Benign Negatives

A detector that keys on single words would be useless in an enterprise, where policies are full of instructions ("never share your password", "submit receipts within 30 days"). Both classes therefore contain legitimate imperative policy language, quoted attack phrases (for example, a security bulletin that quotes a phishing message), markup and inert attacker-target strings. Neither the word "ignore" nor a target token such as "PWNED" separates the classes on its own.

4.5 Partitions and Isolation

The corpus is split 70/10/10/10 (Figure 2, Table 7). Each evaluation partition uses 500 division identifiers that never occur in training, so the classifier was evaluated on 1,000 of the 5,000 divisions. Clean/injected pairs never straddle partitions. The regular test shares the 24 attack families with training; the challenge set contains only the 8 withheld families.

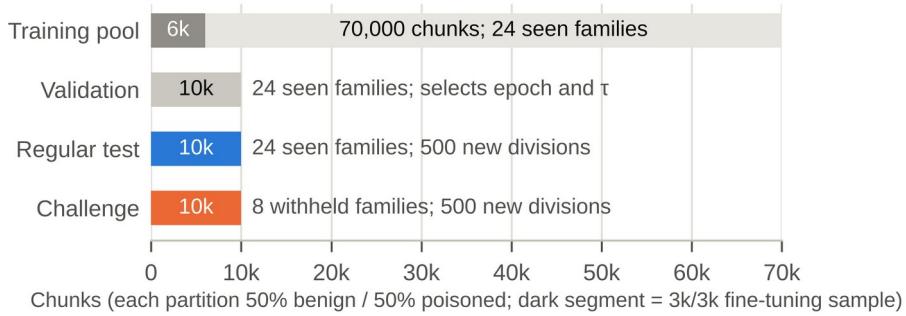


Figure 2: Benchmark partitions. Only a stratified 6,000-chunk sample (dark segment) of the 70,000-chunk training pool was used for fine-tuning. Validation alone selects the checkpoint and threshold, and the challenge set contains only the eight withheld families.

Table 7: Benchmark partitions

Partition	Chunks	Benign / injected	Attack families	Role
Training pool	70,000	35,000 / 35,000	24 seen	6,000 sampled for fine-tuning
Validation	10,000	5,000 / 5,000	24 seen	checkpoint and threshold selection
Regular test	10,000	5,000 / 5,000	24 seen	final evaluation; 500 unseen divisions
Challenge	10,000	5,000 / 5,000	8 withheld	final evaluation; 500 unseen divisions

4.6 Reproducibility and Automated Checks

A manifest records the seed, the finite template inventory and a dataset hash. Earlier datasets are archived before replacement, and the web workbench refuses to attribute an old score to a changed dataset. Unit tests check:

- balanced partitions and reproducible generation;
- that facts survive in the injected chunks;
- cross-split isolation of policies and entities, and isolation of the challenge families;
- that benign chunks contain trigger mentions (the hard negatives);
- metadata round-tripping, threshold selection, and web access to all partitions.

4.7 External Reference Data

The 546 training rows of the public deepset prompt-injections dataset [32] (Apache-2.0; 662 rows in total) are imported into a separate collection. They are not mixed into any synthetic split, because their label definitions differ and they lack clean policy facts. They are reserved for cross-dataset evaluation (Chapter 11).

4.8 From the Legacy Corpus to the 100k Benchmark

The project began with a 200-chunk demonstration corpus. A detector fully fine-tuned on it classified all 60 of its test chunks correctly. A test set that small cannot distinguish detection from template memorisation, which motivated the current benchmark (Table 8).

Table 8: Legacy corpus compared with the current benchmark

Property	Legacy corpus	Current benchmark
Chunks	200 (100 policies × clean/injected)	100,000 (50,000 policies × clean/injected)
Divisions × topics	20 × 5	5,000 divisions; 10 topics
Attack families	6	32 (96 templates)
Test data	60 chunks	10,000 regular + 10,000 challenge
Held-out families	none	8
Detector training	full fine-tuning, 140 chunks	last block + head, 6,000 chunks
Detector result	60 / 60 correct	Chapter 7

5 Layer 1: Detector Design and Training

5.1 Why a Compact Encoder

An encoder-only classifier produces a score in one forward pass, is deterministic, and needs no prompt that an attacker could also target. DistilBERT [6] is a distilled version of BERT that is 40% smaller and 60% faster while retaining about 97% of BERT's language-understanding performance. That makes it a natural choice for CPU inference over every retrieved chunk.

5.2 Windowed Scoring

An injected clause can be a single sentence inside a long chunk, so chunks are not truncated. The tokenizer splits each chunk c into overlapping windows of 256 tokens with a 64-token overlap, forming

the set $\mathcal{W}(c)$. Let $p_\theta(\text{inj} \mid w)$ be the classifier's injection probability for window w. The chunk score and decision are

$$s(c) = \max_{w \in \mathcal{W}(c)} p_\theta(\text{inj} \mid w) \tag{1}$$

$$\hat{y}(c) = 1 \text{ if } s(c) \geq \tau, \text{ otherwise } 0 \tag{2}$$

Taking the maximum keeps a strong local signal from being diluted by long benign text. This deliberately favours recall; its cost, that a single suspicious window can flag a whole chunk, is controlled by the threshold τ.

5.3 Partial Fine-Tuning and Parameter Budget

The embeddings and the first five transformer blocks are frozen. Only the sixth block and the classification head are trained (Table 9). This keeps back-propagation shallow enough for a laptop CPU and limits memory use. It also leaves the pretrained lower layers intact, which we expect to reduce memorisation of templates; this choice has not yet been ablated.

Table 9: Parameter budget of the Layer 1 detector (distilbert-base-uncased, two classes)

Part	Parameters	Trained?
Token + position embeddings, LayerNorm	23,835,648	frozen
Transformer blocks 1–5 (5 × 7,087,872)	35,439,360	frozen
Transformer block 6	7,087,872	trained
Pre-classifier (768 × 768 + 768)	590,592	trained
Classifier (768 × 2 + 2)	1,538	trained
Total	**66,955,010**	**7,680,002 trained (11.47%)**

5.4 Training Data

The fine-tuning set is a stratified sample of 6,000 chunks (3,000 benign, 3,000 injected) drawn from the 70,000-chunk training pool. The sample is stratified across labels, topics, attack families and benign categories. This is 8.6% of the available pool: the run was sized for a CPU budget, and the full pool remains available. Validation and both test partitions are always used in full.

5.5 Training Configuration

Table 10: Training configuration of the recorded run (29 October 2026)

Setting	Value
Base model	distilbert-base-uncased (Hugging Face Transformers [33])
Trainable parameters	7,680,002 of 66,955,010 (11.47%)
Window length / overlap	256 / 64 tokens
Training sample	6,000 of 70,000 chunks (3,000 / 3,000), stratified
Batch size; optimizer steps per epoch	16; 375
Epochs run; epoch selected	3; 1 (validation-based selection with early stopping)
Optimizer	AdamW, learning rate 5×10^{-5}, weight decay 0.01
Learning-rate schedule; clipping	linear with 10% warm-up; gradient-norm clipping at 1.0
Threshold selection	on validation, FPR ≤ 2% → τ = 0.175
Framework; hardware	PyTorch [34] 2.14.1 (CPU build); laptop CPU

Highlighted values are the defaults of the released training code and should be confirmed from the training log.

5.6 Checkpoint and Threshold Selection

After each epoch, the model scores the 10,000 validation chunks. The checkpoint and the operating threshold are chosen on validation data only, to maximise detection subject to a false-positive constraint $\alpha = 2\%$:

$$\tau^{*} = \arg\max_{\tau} \ \mathrm{TPR}_{\mathrm{val}}(\tau) \quad \text{subject to} \quad \mathrm{FPR}_{\mathrm{val}}(\tau) \leq \alpha \tag{3}$$

Validation selected the epoch-1 checkpoint and $\tau = 0.175$. The 2% constraint is a tuning target, not a guarantee on new data, and neither test partition influenced any choice.

5.7 Compute Environment

Training ran entirely on the CPU of a laptop. The laptop has an NVIDIA GTX 1650, but the active environment contained the CPU-only PyTorch build (2.14.1+cpu). A separate CUDA environment was attempted after training, but its download failed when the package host became unreachable, so no GPU result exists. Wall-clock training time was not recorded and should be added if available. The checkpoint and metrics were saved successfully, and earlier checkpoints are archived rather than overwritten.

6 Evaluation Methodology

6.1 Protocol

The selected checkpoint is evaluated once, at $\tau = 0.175$, on all 10,000 regular-test chunks and all 10,000 challenge chunks. Per-chunk predictions are saved (`results/layer1_predictions.csv` and `results/layer1_challenge_predictions.csv`), and the summary goes to `results/layer1_metrics.json`. These numbers measure Layer 1 only, not end-to-end attack prevention.

6.2 Metrics

Table 11: Metrics (TP: injected and flagged; FN: injected and passed; FP: benign and flagged; TN: benign and passed)

Metric	Definition	Meaning in this project
Accuracy	(TP + TN) / N	share of chunks classified correctly
Precision	TP / (TP + FP)	share of alarms that are real injections
Recall (TPR)	TP / (TP + FN)	share of injected chunks detected
False-positive rate	FP / (FP + TN)	share of benign chunks wrongly flagged
Specificity	TN / (TN + FP) = 1 − FPR	share of benign chunks passed unchanged
F1	2 · Precision · Recall / (Precision + Recall)	balance of precision and recall
MCC	(TP·TN − FP·FN) / √((TP+FP)(TP+FN)(TN+FP)(TN+FN))	correlation of prediction and truth (−1 to 1)

6.3 Confidence Intervals

For every proportion $\hat{p} = k/n$, we report a 95% Wilson score interval [35]:

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \qquad z = 1.96 \tag{4}$$

Unlike the normal approximation, the Wilson interval stays informative when $\hat{p}$ is 0 or 1. With zero errors among 5,000 benign chunks, for example, the interval bounds the FPR above by 0.077% rather than collapsing to zero.

6.4 Caveat on Independence

The binomial model assumes independent chunks. Here, clean and injected versions are paired and templates are reused, so the effective sample size is smaller than the chunk count. The intervals therefore describe the sampling of chunks, not of attack templates, and they are optimistic.

7 Detection Results

7.1 Regular Test: Seen Families

The detector classifies all 10,000 regular-test chunks correctly: 5,000 true negatives and 5,000 true positives (Table 12, Figure 3a). The Wilson intervals bound accuracy from below by 99.96% and recall by 99.92%, and they bound the FPR from above by 0.077%. Because the regular test shares attack families and templates with training, this shows that the detector has learned the 24 seen families on unseen policies and divisions. It does not by itself show that injection detection is solved.

Table 12: Detection results at τ = 0.175 (percent; 95% Wilson intervals in brackets)

Metric	Regular test	Challenge
Accuracy	100.00 [99.96, 100.00]	98.09 [97.80, 98.34]
Precision	100.00 [99.92, 100.00]	100.00 [99.92, 100.00]
Recall (TPR)	100.00 [99.92, 100.00]	96.18 [95.61, 96.68]
False-positive rate	0.00 [0.00, 0.077]	0.00 [0.00, 0.077]
Specificity	100.00 [99.92, 100.00]	100.00 [99.92, 100.00]
F1	100.00	98.05
MCC	1.0000	0.9625
TN / FP / FN / TP	5,000 / 0 / 0 / 5,000	5,000 / 0 / 191 / 4,809

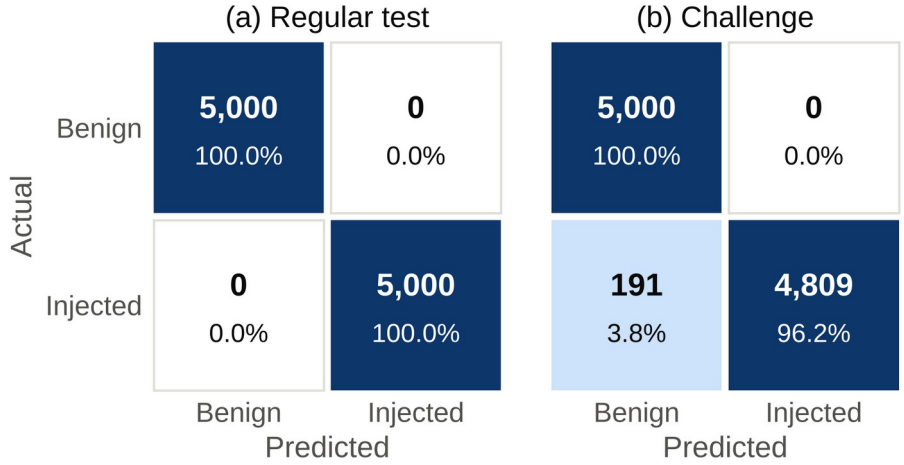


Figure 3: Confusion matrices at τ = 0.175 on (a) the regular test (24 seen families) and (b) the challenge set (8 withheld families). Cells give counts and row percentages.

7.2 Challenge Set: Withheld Families

On the eight withheld families, the detector reaches 98.09% accuracy, 96.18% recall, 98.05% F1 and an MCC of 0.9625 (Figure 3b, Figure 4). All 191 errors are missed injections: none of the 5,000 benign challenge chunks is flagged, so precision and FPR are unchanged from the regular test. Moving to unseen attack styles therefore costs about 3.8 percentage points of recall and no false alarms, at a threshold that was never tuned on these families.

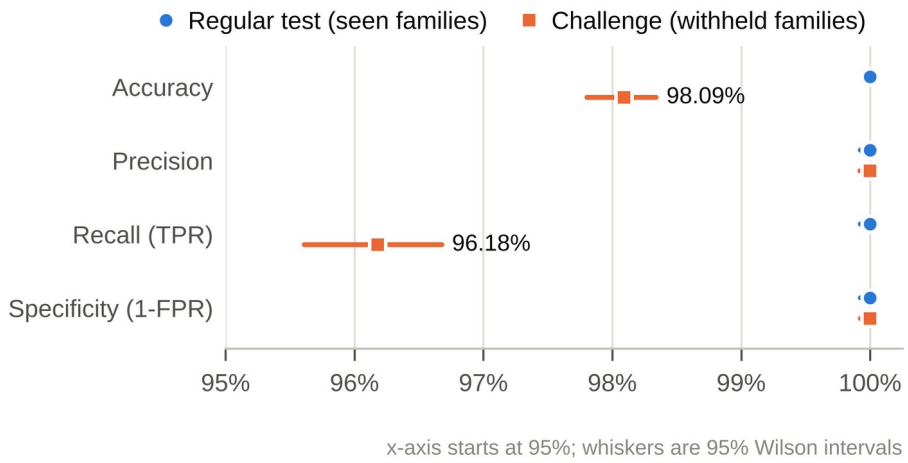


Figure 4: Headline metrics with 95% Wilson intervals. All challenge errors are misses; precision and specificity stay at 100%. The x-axis starts at 95%.

7.3 Per-Family Analysis

Figure 5 and Table 13 break recall down by withheld family. Five families (indirect completion, reasoning lures, polite requests, few-shot imitation and encoded instructions) are detected without a single miss. Three families account for all 191 misses, and multilingual instructions alone for 77.5% of them. Excluding that family, recall on the other seven withheld families is 99.02%. Excluding dependency traps as well, the remaining six families contain 6 misses in 3,766 injected chunks (99.84%).

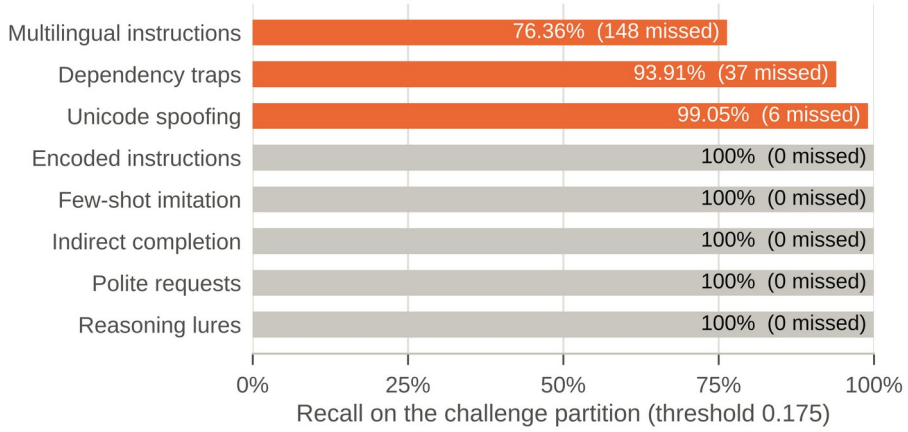


Figure 5: Recall per withheld family on the challenge set (threshold 0.175). Three families account for all 191 misses; multilingual instructions alone account for 148 (77.5%).

Table 13: Recall per withheld family (challenge set)

Family	Injected chunks (n)	Missed	Recall	95% Wilson interval
Multilingual instructions	626	148	76.36%	[72.87, 79.52]
Dependency traps	608	37	93.91%	[91.72, 95.55]
Unicode spoofing	629–634	6	99.05%	≈ [97.9, 99.6]
Encoded instructions	≈ 625	0	100%	≈ [99.4, 100]
Few-shot imitation	≈ 625	0	100%	≈ [99.4, 100]
Indirect completion	≈ 625	0	100%	≈ [99.4, 100]
Polite requests	≈ 625	0	100%	≈ [99.4, 100]
Reasoning lures	≈ 625	0	100%	≈ [99.4, 100]
All withheld families	**5,000**	**191**	**96.18%**	**[95.61, 96.68]**

The n for multilingual instructions and dependency traps are the only integers consistent with the recorded recall. The other family sizes are not in the recorded summary; `plot_from_predictions.m` (Appendix A) prints the exact values from the predictions file.

7.4 Why the Three Families Are Missed

- **Multilingual instructions (recall 76.36%).** The base model was pretrained on English text, and fine-tuning used English chunks, so imperative cues in other languages are weakly represented. This is the largest and most actionable gap.
- **Dependency traps (recall 93.91%).** These injections present the attacker's instruction as a prerequisite of the legitimate task. Their procedural phrasing resembles legitimate policy instructions, which the hard negatives teach the detector to accept. This explanation is inferred from the family's name and its error pattern; inspect the missed chunks to confirm it.
- **Unicode spoofing (recall 99.05%).** Look-alike characters fragment WordPiece tokens, but the surrounding structure of the injection is usually still recognised. Unicode normalisation before scoring should remove most of these misses.

The five fully detected families differ in surface form from the training families, but they still contain text addressed to the assistant that tries to change its behaviour. This suggests, without proving, that the detector responds to that cue rather than only to memorised templates.

7.5 Operating Point and the Cost of Errors

The threshold of 0.175 is low, yet no benign regular-test or challenge chunk crosses it. On this benchmark, therefore, the score distributions of the two classes are well separated, hard negatives included. Real deployments will produce some false positives: PIGuard [9] reports substantial over-defense on natural benign text containing trigger words. The gated design makes such errors cheap. A false positive sends a benign chunk through the sanitizer, which keeps employee-directed instructions and removes only assistant-directed clauses, rather than discarding the document. A false negative, by contrast, delivers the injection to the generator untouched. In this design recall is the critical quantity, which is why the threshold maximises detection under an FPR constraint.

7.6 Summary of Findings

1. On known attack families with new policies and divisions, the detector made no errors on 10,000 chunks.
2. On eight withheld families, recall fell to 96.18% while the false-positive rate stayed at zero.

3. The errors are concentrated: multilingual instructions cause 77.5% of the misses, followed by dependency traps (19.4%) and Unicode spoofing (3.1%).
4. These results were obtained by fine-tuning 11.5% of DistilBERT's parameters on 8.6% of the training pool on a CPU, so the approach is affordable for small teams.
5. All conclusions apply to this synthetic benchmark; external and end-to-end validation are still required.

8 Defense Layer (Layer 2) and End-to-End Harness

8.1 Detector-Gated Clause Sanitization

Only flagged chunks reach Layer 2, which edits a chunk instead of discarding it:

1. **Segmentation.** Structural markers (closing tags, chat-template tokens, role prefixes) become separate units. The remaining text is split into sentences with spaCy [36] and into clauses at connector boundaries.
2. **Rules.** A clause is removed if it matches one of eight strong rule groups (Table 14). Matching also runs on a de-obfuscated copy that undoes leetspeak, hyphenation and letter spacing. Three weak syntactic signals together (for example, imperative mood with a command verb aimed at the output) also remove a clause.
3. **Legitimate context.** Imperative clauses addressed to employees or customers that carry no assistant-directed marker, such as password or receipt rules, are kept and reported as retained instructions.
4. **Reconstruction.** Kept clauses are copied verbatim, only punctuation around removed spans is tidied, and each removal is logged with its reason.

The sanitizer never sees the clean reference text, uses no generative model and is deterministic.

Table 14: Strong rule groups of the clause sanitizer

Rule group	What it removes
Role or system markers	role prefixes such as "System:" or "Assistant:" and chat-template tokens
Delimiter breakouts	closing tags, CDATA ends and fence lines that try to end the data section
Address to the assistant	clauses whose addressee or subject is the AI assistant
Instruction override	attempts to override the question or prior instructions, including common non-English phrasings
Persona switch	attempts to change the assistant's role or task
Output control	dictation of the answer, e.g. what to reply or how to begin
User redirection	telling the assistant to send the user elsewhere (links, numbers, portals)
Exfiltration	Markdown image links and placeholders that could leak data

8.2 Preliminary Verification

The sanitizer was verified on the earlier 200-chunk corpus, which has 20 divisions × 5 topics and 6 attack families placed at the start, middle or end of a chunk, or inline with the fact sentence (Table 15). Two observations stand out. First, the sanitizer restored all 100 injected chunks exactly to their clean text and left all clean chunks unchanged. Second, injections barely reduce retrievability: the poisoned chunk was still ranked first for 95 of 100 questions. Screening must therefore happen *after* retrieval. These checks are not an independent test, because the rules were developed with these cases in view.

The 100,000-chunk benchmark also contains styles (multilingual, Unicode) that the current rules do not cover.

Table 15: Preliminary checks on the 200-chunk legacy corpus

Check	Result
Injected chunks restored exactly to the clean reference text	100 / 100
Clean chunks left unchanged	100 / 100
Out-of-template payloads: attacker-target leaks; lost facts	0; 0 (of 48)
Custom payloads removed	6 / 6
Benign look-alikes kept	10 / 10
Retrieval, clean corpus: target chunk at rank 1	100 / 100
Retrieval, poisoned corpus: injected chunk at rank 1; within top 3	95; 98 (of 100)

Measured with the version of the code in the project repository; re-run on the current local version before citing.

8.3 End-to-End Evaluation Harness

The harness compares four modes on the same questions (Table 16). It reports the following measures:

- **Attack success:** the attacker's target string appears in the answer.
- **Benign utility:** the expected numeric fact appears in the answer.
- **Chunk-level FPR** of the defense.
- **ROUGE-L** [37] between the processed and the clean context.

Failed runs are counted separately and excluded from denominators, the order of modes is randomised, and warm-up calls are excluded. All four measures are proxies. Substring matching can mistake a quoted or refused target for compliance, and ROUGE-L measures overlap, not truth.

Table 16: Defense modes in the evaluation harness

Mode	Behaviour	Extra LLM calls
No defense	retrieved chunks go straight into the prompt	0
Delimiter defense	chunks wrapped in tags with an untrusted-data reminder (Spotlighting-style)	0
LLM-judge defense	the same local LLM judges each chunk; flagged chunks are dropped	one per chunk
Hybrid defense (this project)	**Layer 1 on every chunk, Layer 2 on flagged chunks, plus the reminder**	**0**

8.4 Status

A full four-mode run over one 10,000-question partition with top-3 retrieval requires up to 70,000 generations on a CPU. Only intermediate and interrupted runs exist, so **no end-to-end attack-success numbers are reported**. Because the hybrid mode also adds a reminder, an ablation is needed to separate the reminder's effect from that of the detector and sanitizer.

9 Implementation, Tools and Reproducibility

9.1 Software Stack

Table 17: Software stack

Purpose	Component
Language and environment	Python in a project virtual environment (Windows; administrator rights not needed)
Deep learning	PyTorch [34] 2.14.1 (CPU build); Hugging Face Transformers [33]
Retrieval	sentence-transformers all-MiniLM-L6-v2 [29], [30]; ChromaDB (persistent store, explicit embeddings)
Linguistic processing	spaCy [36], en_core_web_sm
Generation	Ollama serving llama3.2:3b [31]; optional OpenAI client
Interfaces	FastAPI web workbench; Streamlit dashboard
Figures	matplotlib (Python) and MATLAB scripts (Appendix A)

Record the exact library versions (pip freeze) for the paper's reproducibility statement.

9.2 Code Organisation

Table 18: Main modules of the code base

Module	Responsibility
`src/data_generator.py`	benchmark generator (100k default, legacy profile, Hugging Face import)
`src/train_layer1.py`	sampling, partial fine-tuning, validation selection, evaluation on both test sets
`src/layer1_detector.py`	windowed scoring and thresholding at inference
`src/layer2_quarantine.py`	clause sanitizer (segmentation, rules, retention, reconstruction)
`src/embeddings.py`, `src/vector_store.py`	MiniLM embeddings; Chroma collections with explicit vectors
`scripts/index_corpus.py`	incremental indexing; writes `results/index_inventory.json`
`src/llm_client.py`	Ollama and OpenAI clients
`src/rag_pipeline.py`	end-to-end pipeline and command-line interface
`src/evaluate_all.py`, `src/metrics.py`	four-mode harness and metrics
`src/web_server.py`, `web/`, `app.py`	FastAPI workbench and Streamlit dashboard
`tests/`	unit tests

9.3 Inspection Workbench

The web workbench offers a live playground with preset attack scenarios and a browser over six data sources: enterprise corpus, external references, training, validation, regular test and challenge. It also shows saved results and their evidence. For every run it displays the retrieved chunks, the detector score of each chunk, removed and retained clauses with reasons, the final context, the answer and the latency. This supports design goal G4 and is useful for a demonstration track.

9.4 Engineering Safeguards and Tests

- Embeddings are always passed to Chroma explicitly, so the store can never fall back silently to a different default embedder.
- A dataset hash ties every reported score to the data it was computed on.
- Checkpoints and metrics are archived rather than overwritten.
- Offline mode guarantees that no model is downloaded during an experiment.
- Indexing is incremental: unchanged rows are reused and obsolete identifiers are removed in batches.
- Unit tests cover the generator's guarantees, threshold selection and web access to all partitions.

9.5 Reproducing the Results

From the project folder, in a Windows Command Prompt:

```
REM 1. Generate the 100,000-chunk benchmark
.venv\Scripts\python.exe src\data_generator.py --rows 100000 --output-dir data_100k

REM 2. Train and evaluate Layer 1 (6,000-row sample, 3 epochs, CPU)
.venv\Scripts\python.exe src\train_layer1.py --data-dir data_100k --max-train-rows 6000 --
batch-size 16 --epochs 3 --offline

REM 3. Index the corpus for retrieval
.venv\Scripts\python.exe scripts\index_corpus.py --offline --threads 2

REM 4. End-to-end harness (start small; --limit 0 runs every question)
.venv\Scripts\python.exe src\evaluate_all.py --limit 100 --top-k 1 --offline --output-dir
results\rag_regular
.venv\Scripts\python.exe src\evaluate_all.py --split challenge --limit 100 --top-k 1 --
offline --output-dir results\rag_challenge

REM 5. Tests and the workbench
.venv\Scripts\python.exe -m unittest discover -s tests -v
start_demo.bat
```

9.6 Output Files

- `results/layer1_metrics.json`: regular-test and challenge metrics, validation history, threshold, and data counts and hashes.
- `results/layer1_predictions.csv`: one prediction per regular-test chunk.
- `results/layer1_challenge_predictions.csv`: one prediction per challenge chunk, with family labels.
- `results/index_inventory.json`: counts of indexed documents per collection.

10 Limitations

1. **Synthetic data.** The benchmark is template-based (96 templates), paired and English-heavy. The results measure performance on this benchmark, not on natural enterprise documents.
2. **Meaning of "unseen".** The eight families are withheld from fine-tuning only; DistilBERT's pretraining corpus is not controlled.
3. **Statistical dependence.** Paired chunks and shared templates make the confidence intervals optimistic.
4. **One run.** A single seed, trained on 6,000 of the 70,000 available chunks; run-to-run variance is unknown.

5. **Non-adaptive attacker.** An attacker who can query the detector could optimise evasions, as DataSentinel's threat model anticipates [11].
6. **No task context.** The detector sees each chunk without the user's question, so it cannot judge task relevance as Task Shield does [18]. It also does not address false declarative content [8].
7. **Long documents.** Max-over-windows scoring may raise the FPR on long natural documents; this has not been measured.
8. **No external comparison.** There is no comparison yet with PIGuard, PromptShield, PromptArmor, or BIPIA's own defenses on BIPIA data.
9. **Coverage of Layer 2.** A miss by Layer 1 bypasses Layer 2 entirely, the rules cover only recognised patterns, and no end-to-end benchmark has been completed.

11 Recommended Next Experiments

Table 19 ranks the experiments by how much they would strengthen a submission, given the time available.

Table 19: Recommended experiments, in priority order

#	Experiment	Why it matters	Effort
1	Four-mode end-to-end run on 200–500 questions per split (top-3)	turns detector accuracy into attack-success and utility numbers; reviewers will ask for them	1–2 days of CPU
2	Score deepset/prompt-injections and the NotInject benign set with the trained detector	first external evidence of recall and of over-defense	hours
3	Train on the full 70,000-chunk pool with 3 seeds	answers the one-run, 8.6%-of-data objection; gives mean ± s.d.	1–2 days of CPU
4	Add translated versions of *seen* families and apply Unicode normalisation	targets the two main blind spots; the withheld families stay unseen	1 day
5	Measure latency per chunk (detector, sanitizer, LLM judge)	quantifies the no-LLM-call advantage	hours
6	Ablations: partial vs full fine-tuning; max vs mean pooling; reminder on/off	justifies the design choices	1–2 days
7	Simple adaptive attack (paraphrase or obfuscate until $s(c) < \tau$)	first robustness evidence	1–2 days

12 Guidance for Writing the Conference Paper

12.1 Venue Facts: COMSNETS 2027

COMSNETS 2027, the 19th International Conference on COMmunication Systems & NETworkS, takes place in Bengaluru on 5–9 January 2027. The facts below were collected by web search on 29 October 2026. The official site could not be opened directly from the environment that produced this report, so **confirm every date and limit on comsnets.org and in EDAS before planning**.

Table 20: COMSNETS 2027 tracks as reported on 29 October 2026

Track	Deadline (reported)	Length	Review
Main conference	31 August 2026 (closed)	3–8 pages, excluding references	double-blind
Graduate Forum	8 or 9 November 2026 (AoE; two pages disagree); notification 7 Dec; camera-ready 14 Dec	4 pages + 1 page of references + 1-page resume	resume included, so not anonymous
Posters	about 12 November 2026	not yet published for 2027	—
Demos and exhibits	about 10 November 2026	not yet published for 2027	—
Workshops	2027 list not yet confirmed	per workshop	double-blind (2026 rules)

Sources: comsnets.org pages call_for_papers, graduate_forum and submission_guidelines, as reported by search results. The 2027 Graduate Forum page describes eligibility as students of graduate programmes; confirm whether undergraduates qualify.

12.2 Recommended Framing

Title options:

1. *Detect Before You Generate: A Lightweight Retrieval-Time Detector for Indirect Prompt Injection in Local RAG Pipelines* (detection-focused; recommended for the current evidence).
2. *Selective Clause Sanitization with a Lightweight Injection Detector for Local RAG Systems* (better once end-to-end results exist).
3. *How Far Does a CPU-Trained Detector Generalise? Held-Out-Family Evaluation of Indirect Prompt Injection Detection for RAG* (analysis-focused).

Positioning sentence: "Following BIPIA's diagnosis that LLMs confuse retrieved data with instructions, we move the first line of defense out of the generator, to retrieval time, using a detector that is cheap enough for CPU-only local deployment."

Draft abstract (about 230 words) for adaptation

Retrieval-augmented generation (RAG) lets a large language model (LLM) answer from an organisation's documents, but any retrieved chunk may carry instructions planted by an attacker. The BIPIA benchmark found all 25 LLMs it evaluated vulnerable to such indirect prompt injection, and its most effective defense fine-tunes the generator itself, which is out of reach for small models served on CPU-only hardware. We instead screen every retrieved chunk before generation with a lightweight detector, and send only flagged chunks to a rule-based clause sanitizer, so the defense makes no generative call. The detector is a DistilBERT classifier that scores overlapping 256-token windows and takes the maximum injection probability; only its last transformer block and classification head (7.68M of 66.96M parameters) are fine-tuned, on 6,000 chunks, on a laptop CPU. To test generalisation we build a 100,000-chunk synthetic enterprise-policy benchmark with paired clean and injected chunks, 32 attack families and hard benign negatives, and withhold eight families from training and threshold selection. On 10,000 regular-test chunks the detector makes no errors; on 10,000 chunks from the withheld families it attains 98.09% accuracy and 96.18% recall with no false positives. Multilingual injections cause 77.5% of the misses; recall on the other seven withheld families is 99.02%. We discuss what these detector-level results do and do not establish.

Contribution statements (use as written or shorten):

- a 100k-chunk benchmark for retrieval-time injection detection with a family hold-out protocol;
- a CPU-trainable windowed detector (7.68M trainable parameters, 6,000 training chunks);
- a held-out evaluation with per-family failure analysis and uncertainty estimates;
- an inspectable pipeline in which the detector gates a deterministic sanitizer, so the defense makes no generative call.

12.3 Suggested Structure and Page Budget

Table 21: Suggested paper structure (IEEE two-column) with page budget

Section	Full paper (8 pp.)	Graduate Forum (4 pp.)	Material in this report
Abstract and Introduction	1.0	0.6	Ch. 1; draft abstract above
Related work and positioning	0.75	0.3	Ch. 2; Table 3
Threat model and system	0.75	0.5	Ch. 3; Figure 1
Benchmark	0.75	0.4	Ch. 4; Table 6, Figure 2
Detector and training	1.0	0.6	Ch. 5; Eqs. (1)–(3); Table 10
Evaluation setup	0.25	0.1	Ch. 6
Results	1.5	1.1	Ch. 7; Table 12, Figure 3, Figure 5
Defense layer (brief)	0.5	0.2	Ch. 8; Table 15
Limitations, future work, conclusion	0.75	0.2	Ch. 10–11, 13
References	≈ 1 (not counted)	1 (separate page)	References of this report

12.4 Figures and Tables for the Paper

Table 22: Which report items to reuse in the paper

Paper item	Source	Notes
Architecture figure	Figure 1	essential in both formats
Per-family recall figure	Figure 5	the key analysis figure; keep in both formats
Results table	Table 12	essential; can replace the confusion-matrix figure in 4 pages
Confusion matrices	Figure 3	8-page version
Metric intervals figure	Figure 4	optional; duplicates the results table
Partition figure	Figure 2	8-page version; one sentence suffices in 4 pages
Positioning table	Table 3	8-page version; text in 4 pages
Training configuration	Table 10	8-page version; one sentence in 4 pages
Score distribution, ROC, threshold sweep	Appendix A (MATLAB)	add if space allows; generated from the predictions files

All figures are 3.45 in wide (one IEEE column) with 7–8 pt text, and are supplied as PNG (600 dpi) and PDF. Blue always marks the regular test and orange the challenge set; the pair is safe for colour-blind readers.

12.5 Claims and Wording

- Say "withheld from fine-tuning" rather than simply "unseen".
- Say "on this synthetic benchmark" whenever a percentage is quoted.
- Do not claim the first hybrid defense or the first detect-and-remove method (Table 4).
- Do not report end-to-end attack-success numbers until a complete run exists; intermediate runs are not results.
- Cite BIPIA [1] as the base paper for the problem, the threat model and the evaluation philosophy, and PromptArmor [20] as the closest detect-and-remove prior art.

12.6 Items to Verify Before Submission

Table 23: Values to confirm from the project files

Item	Value in this report	Where to check
Learning rate, weight decay, warm-up, clipping	5×10^{-5}; 0.01; 10%; 1.0 (code defaults)	training log or `train_layer1.py` arguments
Window length / overlap	256 / 64 tokens	detector configuration in `config.py`
Name of the eighth withheld family	encoded instructions	generator family list / dataset manifest
Description of dependency traps	instruction framed as a prerequisite	templates; missed chunks in the challenge CSV
Exact family sizes (n)	626, 608, 629–634, ≈ 625	`plot_from_predictions.m` output
Total parameter count	66,955,010 (uncased vocabulary)	`sum(p.numel() for p in model.parameters())`
Layer 2 legacy checks	Table 15	re-run on the current local code
Training time; library versions	not recorded	training log; `pip freeze`
COMSNETS tracks, deadlines, limits	Table 20	comsnets.org and EDAS

12.7 Anonymisation and Artifact Availability

For double-blind tracks, remove all author names, affiliations, acknowledgements and the GitHub link, and write about prior work in the third person. Use "The code, benchmark generator and trained detector will be released upon publication." If reviewers should see code, use an anonymised repository service rather than the project repository. COMSNETS allows an arXiv version, but asks authors not to advertise it on large mailing lists or social media during review.

13 Conclusion

This project moved the first line of defense against indirect prompt injection out of the generator and into the retrieval step. A DistilBERT detector was fine-tuned on a laptop CPU, with 7.68M trainable parameters and 6,000 training chunks. It screens every retrieved chunk and gates a deterministic clause sanitizer, so the defense needs no generative call. On a 100,000-chunk benchmark, it made no errors on 10,000 chunks from known attack families. On 10,000 chunks from eight withheld families, it reached 96.18% recall with zero false positives. The remaining misses are concentrated and actionable: multilingual instructions above all, then dependency traps and Unicode spoofing. Retrieval-time detection is therefore a cheap and inspectable first layer for local RAG. Turning this detector-level evidence into a security claim requires the external, adaptive and end-to-end evaluations listed in Chapter 11.

References

[1] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in *Proc. 31st ACM SIGKDD Conf. Knowledge Discovery and Data Mining V.1 (KDD '25)*, Toronto, ON, Canada, 2025, pp. 1809–1820, doi: 10.1145/3690624.3709179.

[2] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 9459–9474.

[3] OWASP GenAI Security Project, "LLM01:2025 prompt injection," in *OWASP Top 10 for LLM Applications 2025*, Nov. 2024. [Online]. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[4] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, Copenhagen, Denmark, 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[5] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," presented at the *NeurIPS ML Safety Workshop*, 2022, arXiv:2211.09527.

[6] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," presented at the *5th Workshop on Energy Efficient Machine Learning and Cognitive Computing (EMC2), NeurIPS 2019*, arXiv:1910.01108.

[7] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium*, Philadelphia, PA, USA, 2024, pp. 1831–1847.

[8] W. Zou, R. Geng, B. Wang, and J. Jia, "PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models," in *Proc. 34th USENIX Security Symposium*, Seattle, WA, USA, 2025, pp. 3827–3844.

[9] H. Li, X. Liu, N. Zhang, and C. Xiao, "PIGuard: Prompt injection guardrail via mitigating overdefense for free," in *Proc. 63rd Annu. Meeting Assoc. Computational Linguistics (ACL), Vol. 1: Long Papers*, Vienna, Austria, 2025, pp. 30420–30437, doi: 10.18653/v1/2025.acl-long.1468.

[10] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, and D. Wagner, "PromptShield: Deployable detection for prompt injection attacks," in *Proc. 15th ACM Conf. Data and Application Security and Privacy (CODASPY '25)*, Pittsburgh, PA, USA, 2025, pp. 341–352, doi: 10.1145/3714393.3726501.

[11] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, "DataSentinel: A game-theoretic detection of prompt injection attacks," in *Proc. 2025 IEEE Symp. Security and Privacy (SP)*, San Francisco, CA, USA, 2025, pp. 2190–2208, doi: 10.1109/SP61157.2025.00250.

[12] A. Akinrele and S. N. Gowda, "Prompt injection detection is regime-dependent: A deployment-aware evaluation with interpretable structural signals," in *Proc. Fourth UK AI Conference*, Proceedings of Machine Learning Research, vol. 348, Nottingham, UK, 2026, pp. 12–30.

[13] S. Toyer et al., "Tensor Trust: Interpretable prompt injection attacks from an online game," in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, Vienna, Austria, 2024.

[14] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kıcıman, "Defending against indirect prompt injection attacks with spotlighting," in *Proc. Conf. Applied Machine Learning in Information Security (CAMLIS 2024)*, CEUR Workshop Proceedings, vol. 3920, 2024, pp. 48–62.

[15] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *Proc. 34th USENIX Security Symposium*, Seattle, WA, USA, 2025, pp. 2383–2400.

[16] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, "SecAlign: Defending against prompt injection with preference optimization," in *Proc. 2025 ACM SIGSAC Conf. Computer and Communications Security (CCS '25)*, Taipei, Taiwan, 2025, pp. 2833–2847, doi: 10.1145/3719027.3744836.

[17] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," *arXiv preprint arXiv:2404.13208*, 2024.

[18] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The Task Shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents," in *Proc. 63rd Annu. Meeting Assoc. Computational Linguistics (ACL), Vol. 1: Long Papers*, Vienna, Austria, 2025, pp. 29680–29697, doi: 10.18653/v1/2025.acl-long.1435.

[19] E. Debenedetti et al., "Defeating prompt injections by design," in *Proc. 2026 IEEE Conf. Secure and Trustworthy Machine Learning (SaTML)*, Munich, Germany, 2026, pp. 587–618, doi: 10.1109/SaTML68715.2026.00040.

[20] T. Shi et al., "PromptArmor: Simple yet effective prompt injection defenses," *arXiv preprint arXiv:2507.15219*, 2025.

[21] D. Demirol and M. Aydogan, "Balancing security and performance in LLM agents: Spotlight-Guard, a layered defense against indirect prompt injection," *Applied Sciences*, vol. 16, no. 15, Art. no. 7662, 2026, doi: 10.3390/app16157662.

[22] G. Raghuraj, V. N, J. Bharathi, and G. R. Kadavad, "Hybrid defense against indirect prompt injection in RAG: A cross-model benchmark," *International Journal of Artificial Intelligence and Machine Learning*, vol. 6, no. 3, pp. 1148–1156, 2026.

[23] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Assoc. Computational Linguistics: ACL 2024*, Bangkok, Thailand, 2024, pp. 10471–10506, doi: 10.18653/v1/2024.findings-acl.624.

[24] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, Vancouver, BC, Canada, 2024, pp. 82895–82920.

[25] H. Li, R. Wen, S. Shi, N. Zhang, Y. Vorobeychik, and C. Xiao, "AgentDyn: Are your agent security defenses deployable in real-world dynamic environments?" *arXiv preprint arXiv:2602.03117*, 2026.

[26] X. Liang et al., "SafeRAG: Benchmarking security in retrieval-augmented generation of large language model," in *Proc. 63rd Annu. Meeting Assoc. Computational Linguistics (ACL), Vol. 1: Long Papers*, Vienna, Austria, 2025, pp. 4609–4631, doi: 10.18653/v1/2025.acl-long.230.

[27] H. Guo and Z. Wei, "Hidden-in-plain-text: A benchmark for social-web indirect prompt injection in RAG," in *Proc. ACM Web Conference 2026 (WWW '26)*, 2026, pp. 8337–8340, doi: 10.1145/3774904.3792853.

[28] X. Cao and Y. Bao, "A systematic survey of prompt injection attacks and defenses in LLM-integrated applications and agentic systems," in *Proc. 2026 12th Int. Conf. Big Data and Information Analytics (BigDIA)*, 2026, pp. 16–26.

[29] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, 2019, pp. 3982–3992, doi: 10.18653/v1/D19-1410.

[30] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 5776–5788.

[31] A. Grattafiori et al., "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[32] deepset, "prompt-injections," *Hugging Face Datasets*, 2023. [Online]. Available: https://huggingface.co/datasets/deepset/prompt-injections

[33] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. 2020 Conf. Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45, doi: 10.18653/v1/2020.emnlp-demos.6.

[34] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, Vancouver, BC, Canada, 2019, pp. 8024–8035.

[35] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927, doi: 10.1080/01621459.1927.10502953.

[36] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spaCy: Industrial-strength natural language processing in Python," Zenodo, 2020, doi: 10.5281/zenodo.1212303.

[37] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, Barcelona, Spain, 2004, pp. 74–81.

Appendix A: Reproducing the Figures

A.1 Figures in this Report (Python)

The figures were generated with Python and matplotlib from the recorded numbers, so every figure and table agrees by construction:

```
cd paper\build
python paper_numbers.py      & REM writes numbers.json (counts, intervals, parameter
counts)
python make_figures.py       & REM writes ..\figures\fig1..fig5 (.png 600 dpi, .pdf, .svg)
python make_equations.py     & REM writes ..\figures\eq1..eq4 (.png)
```

A.2 MATLAB: Step by Step

The MATLAB scripts need no toolboxes and were tested end to end (in GNU Octave 10.3) on files with the same structure as the project's outputs.

1. Copy the `matlab` folder into the project, e.g. `<project>\paper\matlab`, so that the results folder is `..\..\results`. Otherwise, edit `RESULTS_DIR` and `METRICS_JSON` at the top of the scripts.
2. Start MATLAB and set the Current Folder to the `matlab` folder.
3. Run `make_report_figures`. It writes Figures 2–5 to `figures_matlab\` as 600-dpi PNG and vector PDF. No data files are needed.
4. Run `plot_from_predictions`. It reads both predictions files, prints the confusion matrices at τ = 0.175 (they must equal Table 12) and the per-family table with exact n and Wilson intervals. It then draws Figures A1–A5: score distributions, ROC, precision–recall, threshold sweep and per-family recall. If a column is not detected, it lists the available columns; set `LABEL_COLUMN`, `SCORE_COLUMN` or `FAMILY_COLUMN` and run again.
5. Run `plot_validation_history` to plot the per-epoch validation curves stored in `layer1_metrics.json` (Figure A6). If it cannot find them, set `HISTORY_FIELD` to the field name it prints.
6. Insert the PNG files into Word (Insert → Pictures) at 3.5 in width for a two-column paper. In LaTeX, use the PDF files with `\includegraphics[width=\columnwidth]{...}`.

Table 24: Figures, scripts and data

Figure	MATLAB script	Data needed
Figure 1 (architecture)	— (diagram; reuse the PNG or redraw)	—
Figure 2–Figure 5	`make_report_figures.m`	none (recorded numbers in the script)
A1 score distributions	`plot_from_predictions.m`	both predictions CSV files
A2 ROC curves, A3 precision–recall	`plot_from_predictions.m`	both predictions CSV files
A4 threshold sweep	`plot_from_predictions.m`	challenge predictions CSV
A5 per-family recall (exact n)	`plot_from_predictions.m`	challenge predictions CSV with family column
A6 validation history	`plot_validation_history.m`	`layer1_metrics.json`

A.3 Listing: make_report_figures.m

```
%% make_report_figures.m
% Rebuilds Figures 2-5 of the report from the recorded Layer 1 results
% (run of 29 Oct 2026). No data files or toolboxes are needed.
%
% Steps: open this folder in MATLAB, then press Run (or type
% make_report_figures in the Command Window). PNG (600 dpi) and PDF files
% are written to ./figures_matlab. Figure 1 (the architecture diagram) is a
% drawing, not a chart; reuse the supplied PNG or redraw it in a diagram tool.

OUT_DIR = fullfile('.', 'figures_matlab');

% ---- recorded results -----------------------------------------------------
PARTITION_NAMES = {'Training pool', 'Validation', 'Regular test', 'Challenge'};
PARTITION_SIZE  = [70000 10000 10000 10000];   % chunks, each 50% benign
TRAIN_SAMPLE    = 6000;                        % 3,000 benign + 3,000 injected
THRESHOLD       = 0.175;

CM_REGULAR   = [5000 0; 0 5000];      % rows: actual benign/injected; cols: predicted
CM_CHALLENGE = [5000 0; 191 4809];

FAMILY_NAMES  = {'Multilingual instructions', 'Dependency traps', 'Unicode spoofing', ...
                 'Encoded instructions', 'Few-shot imitation', 'Indirect completion', ...
                 'Polite requests', 'Reasoning lures'};
FAMILY_MISSED = [148 37 6 0 0 0 0 0];
FAMILY_RECALL = [0.7636 0.9391 0.9905 1 1 1 1 1];

% ---- palette (validated for colour-vision deficiency on white) -----------
BLUE   = [42 120 214] / 255;    % regular test
ORANGE = [235 104 52] / 255;    % challenge
INK    = [11 11 11] / 255;
INK2   = [82 81 78] / 255;
GRID   = [225 224 217] / 255;
G_LIGHT = [230 229 223] / 255;
G_MID   = [201 199 191] / 255;
G_DARK  = [143 141 134] / 255;
SEQ = [255 255 255; 205 226 251; 158 197 244; 109 167 236; 57 135 229; 37 106 191; 24 79 149; 13
54 107] / 255;
TXT = {'FontName', 'Arial', 'FontSize', 7};

%% Figure 2 - benchmark partitions
fig = new_figure(3.45, 1.35);
ax = axes('Parent', fig, 'Position', [0.25 0.30 0.72 0.66]);
hold(ax, 'on');
colours = {G_LIGHT, G_MID, BLUE, ORANGE};
notes = {'', '24 seen families; selects epoch and \tau', '24 seen families; 500 new
divisions', ...
         '8 withheld families; 500 new divisions'};
for i = 1:4
    y = 5 - i;
    rectangle('Parent', ax, 'Position', [0, y - 0.29, PARTITION_SIZE(i) / 1000, 0.58], ...
        'FaceColor', colours{i}, 'EdgeColor', 'none');
    if i == 1
        rectangle('Parent', ax, 'Position', [0, y - 0.29, TRAIN_SAMPLE / 1000, 0.58], ...
            'FaceColor', G_DARK, 'EdgeColor', 'none');
        text(3, y, '6k', 'Parent', ax, 'Color', 'w', 'HorizontalAlignment', 'center', TXT{:});
        text(38, y, '70,000 chunks; 24 seen families', 'Parent', ax, 'Color', INK, ...
            'HorizontalAlignment', 'center', TXT{:});
    else
        if i >= 3
            label_colour = 'w';
        else
            label_colour = INK;
        end
        text(5, y, '10k', 'Parent', ax, 'Color', label_colour, 'HorizontalAlignment', 'center',
TXT{:});
        text(11.2, y, notes{i}, 'Parent', ax, 'Color', INK2, TXT{:});
    end
end
set(ax, 'YTick', 1:4, 'YTickLabel', fliplr(PARTITION_NAMES), 'XLim', [0 70], 'YLim', [0.45 4.45],
...
    'XTick', 0:10:70, 'XTickLabel', {'0', '10k', '20k', '30k', '40k', '50k', '60k', '70k'});
style_axes(ax);
set(ax, 'XGrid', 'on', 'GridColor', GRID, 'Layer', 'bottom');
xlabel(ax, 'Chunks (each partition 50% benign / 50% poisoned)', 'Color', INK2, TXT{:});
save_figure(fig, OUT_DIR, 'fig2_partitions');

%% Figure 3 - confusion matrices
fig = new_figure(3.45, 1.70);
mats = {CM_REGULAR, CM_CHALLENGE};
titles = {'(a) Regular test', '(b) Challenge'};
for p = 1:2
    ax = subplot(1, 2, p, 'Parent', fig);
```

```
    hold(ax, 'on');
    M = mats{p};
    for i = 1:2
        for j = 1:2
            v = M(i, j);
            share = v / sum(M(i, :));
            if v == 0
                idx = 1;
            else
                idx = max(2, min(8, round(share * 7) + 1));
            end
            if idx >= 5
                tc = 'w';
            else
                tc = INK;
            end
            edge = 'none';
            if idx == 1
                edge = GRID;
            end
            rectangle('Parent', ax, 'Position', [j - 1 + 0.02, 2 - i + 0.02, 0.96, 0.96], ...
                'FaceColor', SEQ(idx, :), 'EdgeColor', edge);
            text(j - 0.5, 2 - i + 0.62, sprintf('%s', commas(v)), 'Parent', ax, 'Color', tc, ...
                'FontWeight', 'bold', 'HorizontalAlignment', 'center', 'FontName', 'Arial',
'FontSize', 8);
            text(j - 0.5, 2 - i + 0.33, sprintf('%.1f%%', 100 * share), 'Parent', ax, 'Color',
tc, ...
                'HorizontalAlignment', 'center', TXT{:});
        end
    end
    axis(ax, 'equal');
    set(ax, 'XLim', [0 2], 'YLim', [0 2], 'XTick', [0.5 1.5], 'XTickLabel', {'Benign',
'Injected'}, ...
        'YTick', [0.5 1.5], 'YTickLabel', {'Injected', 'Benign'}, 'TickLength', [0 0]);
    style_axes(ax);
    set(ax, 'XColor', INK2, 'YColor', INK2);
    title(ax, titles{p}, 'FontWeight', 'normal', 'Color', INK, TXT{:});
    xlabel(ax, 'Predicted', TXT{:});
    if p == 1
        ylabel(ax, 'Actual', TXT{:});
    else
        set(ax, 'YTickLabel', {'', ''});
    end
end
save_figure(fig, OUT_DIR, 'fig3_confusion');

%% Figure 4 - headline metrics with 95% Wilson intervals
metric_names = {'Accuracy', 'Precision', 'Recall (TPR)', 'Specificity (1-FPR)'};
fig = new_figure(3.45, 1.65);
ax = axes('Parent', fig, 'Position', [0.30 0.22 0.62 0.62]);
hold(ax, 'on');
series = {CM_REGULAR, CM_CHALLENGE};
colours = {BLUE, ORANGE};
markers = {'o', 's'};
offsets = [0.14 -0.14];
handles = zeros(1, 2);
for k = 1:2
    M = series{k};
    TN = M(1, 1); FP = M(1, 2); FN = M(2, 1); TP = M(2, 2);
    kk = [TP + TN, TP, TP, TN];
    nn = [sum(M(:)), TP + FP, TP + FN, TN + FP];
    [lo, hi] = wilson_ci(kk, nn);
    v = kk ./ nn;
    for m = 1:4
        y = 5 - m + offsets(k);
        plot(ax, 100 * [lo(m) hi(m)], [y y], '-', 'Color', colours{k}, 'LineWidth', 1.4);
        h = plot(ax, 100 * v(m), y, markers{k}, 'MarkerFaceColor', colours{k}, ...
            'MarkerEdgeColor', 'w', 'MarkerSize', 5, 'LineWidth', 0.8);
        if m == 1
            handles(k) = h;
        end
        if k == 2 && v(m) < 0.995
            text(100 * hi(m) + 0.08, y, sprintf('%.2f%%', 100 * v(m)), 'Parent', ax, 'Color',
INK, TXT{:});
        end
    end
end
set(ax, 'YTick', 1:4, 'YTickLabel', fliplr(metric_names), 'XLim', [95 100.25], 'YLim', [0.45
4.55], ...
    'XTick', 95:100, 'XTickLabel', {'95%', '96%', '97%', '98%', '99%', '100%'});
style_axes(ax);
set(ax, 'XGrid', 'on', 'GridColor', GRID);
legend(handles, {'Regular test (seen families)', 'Challenge (withheld families)'}, ...
    'Location', 'northoutside', 'Orientation', 'horizontal', 'Box', 'off', TXT{:});
save_figure(fig, OUT_DIR, 'fig4_metrics_ci');
```

```
%% Figure 5 - recall per withheld family
[~, order] = sortrows([FAMILY_RECALL(:), (1:numel(FAMILY_RECALL))']);
fig = new_figure(3.45, 2.05);
ax = axes('Parent', fig, 'Position', [0.36 0.18 0.61 0.79]);
hold(ax, 'on');
nf = numel(order);
for r = 1:nf
    i = order(r);
    y = nf + 1 - r;
    if FAMILY_MISSED(i) > 0
        c = ORANGE;
        label = sprintf('%.2f%%  (%d missed)', 100 * FAMILY_RECALL(i), FAMILY_MISSED(i));
        tc = 'w';
    else
        c = G_MID;
        label = '100%  (0 missed)';
        tc = INK;
    end
    rectangle('Parent', ax, 'Position', [0, y - 0.31, 100 * FAMILY_RECALL(i), 0.62], ...
        'FaceColor', c, 'EdgeColor', 'none');
    text(100 * FAMILY_RECALL(i) - 1.5, y, label, 'Parent', ax, 'Color', tc, ...
        'HorizontalAlignment', 'right', TXT{:});
end
set(ax, 'YTick', 1:nf, 'YTickLabel', fliplr(FAMILY_NAMES(order)), 'XLim', [0 100], 'YLim', [0.4
nf + 0.6], ...
    'XTick', 0:25:100, 'XTickLabel', {'0%', '25%', '50%', '75%', '100%'});
style_axes(ax);
set(ax, 'XGrid', 'on', 'GridColor', GRID, 'Layer', 'top');
xlabel(ax, sprintf('Recall on the challenge partition (threshold %.3f)', THRESHOLD), TXT{:});
save_figure(fig, OUT_DIR, 'fig5_family_recall');

fprintf('Done. Figures are in %s\n', OUT_DIR);
```

A.4 Listing: wilson_ci.m

```
function [lo, hi] = wilson_ci(k, n, z)
%WILSON_CI  95% Wilson score interval for k successes out of n trials.
%   [lo, hi] = wilson_ci(k, n) returns proportions in [0, 1]. Works on
%   vectors element-wise. Pass z to change the confidence level.
if nargin < 3
    z = 1.959963984540054;
end
p = k ./ n;
den = 1 + z.^2 ./ n;
centre = (p + z.^2 ./ (2 .* n)) ./ den;
half = z .* sqrt(p .* (1 - p) ./ n + z.^2 ./ (4 .* n.^2)) ./ den;
lo = max(0, centre - half);
hi = min(1, centre + half);
end
```

Appendix B: Equations in LaTeX

For the paper, the equations can be typeset natively:

```
% (1) chunk score
s(c) = \max_{w \in \mathcal{W}(c)} p_\theta(\mathrm{inj} \mid w)
% (2) decision
\hat{y}(c) = \mathbb{1}\left[ s(c) \ge \tau \right]
% (3) threshold selection on validation data
\tau^{*} = \arg\max_{\tau} \mathrm{TPR}_{\mathrm{val}}(\tau) \quad \text{s.t.} \quad \
mathrm{FPR}_{\mathrm{val}}(\tau) \le \alpha
% (4) Wilson score interval
\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}
{1 + \frac{z^2}{n}}, \qquad z = 1.96
```

In Word, use Insert → Equation and type the same expressions in LaTeX mode (Microsoft 365 accepts LaTeX syntax).

Appendix C: Recorded Numbers and Derivations

C.1 Confusion Matrices (Rows: Actual; Columns: Predicted)

	Regular test		Challenge	
	benign	injected	benign	injected
actual benign	5,000	0	5,000	0
actual injected	0	5,000	191	4,809

C.2 Derived Quantities

- Challenge recall = 4,809 / 5,000 = 96.18%; F1 = 2 × 1 × 0.9618 / 1.9618 = 98.05%; MCC = 4,809 / √(4,809 × 5,191) = 0.9625.
- Multilingual family size: the only integer n with $1 - 148/n$ rounding to 76.36% is n = 626. Dependency traps: $1 - 37/n$ = 93.91% gives n = 608. Unicode spoofing: $1 - 6/n$ = 99.05% holds for n = 629 to 634.
- Recall without the multilingual family: (4,809 − 478) / (5,000 − 626) = 4,331 / 4,374 = 99.02%. Without multilingual instructions and dependency traps: 3,760 / 3,766 = 99.84%.
- Shares of the 191 misses: 148 (77.5%), 37 (19.4%), 6 (3.1%).
- Optimizer steps: ⌈6,000 / 16⌉ = 375 per epoch; 3 epochs = 1125 steps.
- DistilBERT parameters: embeddings 30,522 × 768 + 512 × 768 + 2 × 768 = 23,835,648; one block 4 × (768^2 + 768) + 2 × 768 + (768 × 3,072 + 3,072) + (3,072 × 768 + 768) + 2 × 768 = 7,087,872; head (768^2 + 768) + (768 × 2 + 2) = 592,130; total 66,955,010.